

Guía visual de la zona de personalización

Dashboard por bloques en main.py

Qué controla	Dónde vive	Para qué sirve
Layout, columnas, widgets y bloques	main.py Zona editable + construcción final	Modificar el dashboard sin tocar todo el runtime

Vista general en 1 minuto

Si quieres cambiar...	Debes tocar...
La estructura del dashboard	DEFAULT_TEMPLATE_CONFIG y layout_rows
Qué columnas usa el CSV	DEFAULT_COLUMN_CANDIDATES
Qué widgets aparecen y en qué orden	DEFAULT_BLOCKS + bloque_fila_*
Construcción desde JSON	config.json con dashboard_modular + FACTORY_REGISTRY

Regla mental rápida: el layout define qué celdas existen, y cada bloque decide qué widget cae en cada celda.

Mapa del sistema: dónde vive cada pieza

Antes de configurar bloques, ubica el rol de cada carpeta y archivo para no editar en el lugar equivocado.

Ruta / archivo	Para qué sirve	Tipo de responsabilidad
FORMS/	Contiene el ejemplo de implementación con vista_py.scx.	Referencia visual/integración VFP
python310_embed/	Aquí vive el ejecutable de Python con configuraciones ya definidas para su funcionamiento.	Runtime embebido
python310_embed/	También contiene los códigos del repositorio adaptados para compatibilidad con VFP.	Compatibilidad e integración
graficos.py	Métodos/fábricas para gráficas y serialización ChartWidget -> legacy widget.	Capa de gráficas
kpi.py	Métodos/fábricas para KPI y serialización KPIWidget -> legacy	Capa de indicadores

	widget.	
tablas.py	Funciones para tablas y serialización TableWidget -> legacy widget.	Capa tabular
main.py	Aquí se genera la construcción, vive la zona editable y se ensamblan los bloques.	Centro de orquestación
graficador_app.js	JS necesario para la ejecución del dashboard.	Lógica cliente
graficador_body.html	Da estructura al body del dashboard.	HTML base
graficador_head.html	Da estructura del head y el CSS.	Head + estilos
config.json	Sistema alternativo para configurar por JSON cuando no se quieren usar bloques.	Entrada declarativa

Flujo visual de construcción

1	DEFAULT_TEMPLATE_CONFIG	Define el esqueleto del template y las filas/columnas.
2	DEFAULT_COLUMN_CANDIDATES	Propone nombres posibles de columnas reales del CSV.
3	_default_columns(...)	Resuelve el mapa final de columnas.
4	bloque_fila_*	Cada bloque crea widgets y los manda a una celda.
5	DEFAULT_BLOCKS	Decide qué bloques se ejecutan y en qué orden.
6	_apply_default_blocks(...)	Recorre y ejecuta cada bloque.
7	_build_default_template(...)	Arma el template final cuando no se usa JSON modular.
8	_build_template_from_modular_spec(...)	Arma el template desde config.json usando factories.
9	build_dashboard_config(...)	Integra el template al runtime del dashboard.

Layout y mapa de celdas

Ejemplo actual: layout_rows = [4, 3, 2, 1]. Esto significa que el dashboard se reparte así:

cell_1_1	cell_1_2	cell_1_3	cell_1_4
cell_2_1	cell_2_2	cell_2_3	
cell_3_1	cell_3_2		
cell_4_1			

Importante: si cambias layout_rows, revisa todos los builder.cell(fila, columna) de los bloques, porque una celda puede dejar de existir o quedar ocupada por otro widget.

Mapa de columnas: de nombre lógico a nombre real

DEFAULT_COLUMN_CANDIDATES permite adaptar el dashboard a distintos CSV sin reescribir todos los widgets.

Variable lógica	Candidatos en CSV	Para qué se usa
apertura	apertura, fecha	Fecha principal del dashboard
total	total, subtotal	Monto principal de venta
subtotal	subtotal, total	Valor alterno de venta
personas	personas	Agrupación para comparativos
total_articulos	totalarticulos, total	Cantidad total de artículos
pagado	pagado, cancelado	Estado o bandera de pago
empresa	empresa	Campo descriptivo para la tabla
cancelado	cancelado	Estado alterno de cancelación

Idea clave: las claves del diccionario no tienen que existir tal cual en el CSV; basta con que exista al menos uno de sus candidatos.

Bloques por fila: qué construye cada uno

bloque_fila_1_kpis · Fila 1

Qué hace	Crea 4 KPI en cell_1_1 a cell_1_4
Qué usa	Usa total, apertura y personas
Qué sueles cambiar	Cambias title, format, accent_color, id

bloque_fila_2_grafica_izq · Fila 2	
Qué hace	Gráfica bar en cell_2_1
Qué usa	Usa personas y total
Qué sueles cambiar	Cambias aggregation, top_n, title, id

bloque_fila_2_kpi_centro · Fila 2	
Qué hace	KPI de artículos vendidos en cell_2_2
Qué usa	Usa total_articulos
Qué sueles cambiar	Cambias format, accent_color, title, id

bloque_fila_2_grafica_der · Fila 2	
Qué hace	Tendencia en cell_2_3
Qué usa	Usa apertura y total
Qué sueles cambiar	Cambias analysis_mode, date_column, aggregation, date_granularity

bloque_fila_3_grafica_izq · Fila 3	
Qué hace	Barra lateral en cell_3_1
Qué usa	Usa personas y subtotal
Qué sueles cambiar	Cambias aggregation, top_n, title, id

bloque_fila_3_grafica_der · Fila 3	
Qué hace	Dona de estado de pago en cell_3_2
Qué usa	Usa pagado y apertura

Qué sueles cambiar	Cambias aggregation, date_column, top_n, title
bloque_fila_4_tabla_detalle · Fila 4	
Qué hace	Tabla final en cell_4_1
Qué usa	Usa empresa, apertura, personas, subtotal, total_articulos, total, pagado y cancelado
Qué sueles cambiar	Cambias sort_by, sort_dir, limit, title, id

FACTORY_REGISTRY: catálogo de widgets disponibles

Grupo	Fábricas registradas
KPIs	kpi_sum, kpi_avg, kpi_count, kpi_min, kpi_max
Gráficas	bar, linea, puntos, barra_lateral, pie, dona, polar, radar
Tablas	tabla

Edita este registro cuando agregues una fábrica nueva o quieras habilitar un nombre nuevo para el flujo JSON.

¿Qué cambia cuando usas JSON?

Sin JSON modular	Se usa _build_default_template(...) y los bloque_fila_*.
Con dashboard_modular.enabled = true	Se construye desde config.json usando FACTORY_REGISTRY.
Dónde se traduce	En _build_template_from_modular_spec(...) y luego build_dashboard_config(...).
Conclusión práctica	La zona por bloques sigue sirviendo como fallback, plantilla base o referencia visual.

Variables y opciones disponibles

Esta sección resume para qué sirve cada variable importante y qué valores se le pueden pasar en la práctica.

Variables del template / layout

Variable	Para qué sirve	Opciones / valores
template_id	Identificador interno del template activo.	Texto libre tipo slug, por ejemplo ventas-modular.
template_name	Nombre visible del template.	Texto libre.
title	Título mostrado en el dashboard.	Texto libre.
description	Descripción del dashboard.	Texto libre.
layout_rows	Define cuántas columnas tendrá cada fila.	Lista de enteros. Cada valor se normaliza entre 1 y 4. Ejemplo: [4,3,2,1].

Variables de resolución de columnas

Variable	Para qué sirve	Opciones / valores
DEFAULT_DATE_CANDIDATES	Posibles nombres para la columna de fecha.	Tupla/lista de textos. Ejemplo: apertura, fecha, date.
DEFAULT_COLUMN_CANDIDATES	Mapa de columna lógica -> candidatos reales del CSV.	Dict[str, tuple[str,...]].
column / x_column / y_column / date_column / sort_by	Campos que pasan por ColumnResolver.resolve(...).	Puedes usar nombre real o alias lógico mientras exista en el CSV.
columns	Lista de columnas para tabla.	Lista de strings; se limpian duplicados y vacíos.
filters	Filtros por widget o globales.	Lista de dicts con column, operator y value.

Variables comunes de gráficas

Variable	Para qué sirve	Opciones / valores
chart_type	Tipo final enviado al widget legado.	bar, line, scatter, horizontalBar, pie, doughnut, polarArea, radar.
analysis_mode	Modo analítico de la gráfica.	categorías, tendencia,

		composicion, scatter.
aggregation	Agregación usada al resumir datos.	sum, avg, count, min, max. En scatter suele resolverse a count por defecto.
date_granularity	Agrupar fechas cuando hay tendencia.	day, month, year u otros valores que soporte tu runtime.
top_n	Recorta cantidad de categorías.	Entero >= 1.
point_limit	Límite de puntos a renderizar.	Entero >= 1.
id	ID del widget.	Texto libre; si no se define se genera con slug.
title	Título visible del widget.	Texto libre.
cell_id	Celda donde se renderiza el widget.	Formato cell_fila_columna, por ejemplo cell_2_3.

Variables de KPI

Variable	Para qué sirve	Opciones / valores
column	Columna sobre la que opera el KPI.	Nombre real o resoluble.
aggregation	Agregación del KPI.	sum, avg, count, min, max.
format	Formato visual del valor.	currency, integer, number, o el que soporte el frontend.
accent_color	Color de acento del KPI.	Hex, por ejemplo #1d4ed8.
title / id / cell_id	Metadatos del widget KPI.	Texto libre y celda válida.

Variables de tabla

Variable	Para qué sirve	Opciones / valores
columns	Columnas visibles en la tabla.	Lista de strings.
limit	Cantidad máxima de registros.	Entero >= 1.
sort_by	Columna de ordenamiento.	Nombre real o resoluble.

sort_dir	Dirección de ordenamiento.	asc o desc.
filters	Filtros aplicados a la tabla.	Lista de dicts con column, operator y value.

Variables de filtros y fechas

Variable	Para qué sirve	Opciones / valores
operator	Operador dentro de cada filtro.	eq por defecto; también puedes pasar el operador que entienda tu runtime.
value	Valor del filtro.	Texto, número, fecha o bandera.
date_range.enabled	Activa o desactiva filtro de rango.	true / false.
date_range.column	Columna de fecha usada para rango.	Nombre real o resoluble.
date_range.start / end	Inicio y fin del rango.	Texto de fecha; vacío si no se usa.
dashboard_modular.enabled	Activa construcción desde JSON.	true / false.

Resumen práctico de factories

Factory	Tipo	Parámetros base	Notas útiles
kpi_sum	KPI	column, title, cell_id, id, format, accent_color	aggregation fija = sum
kpi_avg	KPI	column, title, cell_id, id, format, accent_color	aggregation fija = avg
kpi_count	KPI	column, title, cell_id, id, format, accent_color	aggregation fija = count
kpi_min	KPI	column, title, cell_id, id, format, accent_color	aggregation fija = min
kpi_max	KPI	column, title, cell_id, id, format, accent_color	aggregation fija = max
bar	Chart	x_column, y_column, title, cell_id + kwargs	chart_type bar, mode categorias
barra_lateral	Chart	x_column, y_column,	chart_type horizontalBar

		title, cell_id + kwargs	
linea	Chart	x_column, y_column, title, cell_id + kwargs	si pasas date_column, puede volverse tendencia
puntos	Chart	x_column, y_column, title, cell_id + kwargs	fuerza scatter/scatter
pie	Chart	x_column, y_column, title, cell_id + kwargs	mode composicion
dona	Chart	x_column, y_column, title, cell_id + kwargs	chart_type doughnut
polar	Chart	x_column, y_column, title, cell_id + kwargs	chart_type polarArea
radar	Chart	x_column, y_column, title, cell_id + kwargs	chart_type radar
tabla	Table	columns, title, cell_id, id, limit, sort_by, sort_dir, filters	Serializa a type table

Configuración de un nuevo bloque

Usa esta receta cuando quieras agregar una nueva pieza al dashboard por bloques sin romper el armado actual.

1. Verifica el layout	Confirma que layout_rows tenga una celda disponible donde vas a colocar el nuevo widget.
2. Elige la celda	Usa builder.cell(fila, columna). Si la celda no existe o ya está usada, builder.add(...) fallará.
3. Resuelve columnas	Dentro del bloque usa columns = _default_columns(resolver) o resolver.resolve(...) para no amarrarte a un CSV exacto.
4. Construye el widget	Llama una factory de graficos.py, kpi.py o tablas.py, o una nueva factory registrada.
5. Agrega el widget	Pásalo a builder.add(widget) o crea directo con builder.add(factory(...)).
6. Registra el bloque	Incluye la función en DEFAULT_BLOCKS en el orden

	en que debe ejecutarse.
7. Si es factory nueva	Agrégalas a FACTORY_REGISTRY para que también funcione desde config.json.
8. Prueba generación	Corre generar_dashboard_csv(...) y valida la celda, columnas y serialización final.

Plantilla mínima de un bloque nuevo

```
def bloque_fila_3_nueva(builder: DashboardBuilder, resolver: ColumnResolver) -> None:
    columns = _default_columns(resolver)
    builder.add(
        bar(
            columns["empresa"],
            columns["total"],
            "Venta por empresa",
            builder.cell(3, 2),
            id="chart_venta_empresa",
            aggregation="sum",
            top_n=10,
        )
    )

DEFAULT_BLOCKS.append(bloque_fila_3_nueva)
```

Checklist antes de darlo por bueno

- La celda existe según layout_rows.
- La celda no está repetida en otro bloque.
- Las columnas realmente existen o son resolubles por DEFAULT_COLUMN_CANDIDATES.
- La factory usada existe en el archivo correcto y serializa con to_legacy_widget(...).
- Si el bloque debe vivir también en JSON, la factory está en FACTORY_REGISTRY.
- El widget final respeta id, title, aggregation, formatos y filtros esperados.

Resumen mental final

Piensa esta zona como un tablero Lego: DEFAULT_TEMPLATE_CONFIG define la base, DEFAULT_COLUMN_CANDIDATES dice qué piezas del CSV usar, las factories crean los widgets, los bloque_fila_* los colocan y DEFAULT_BLOCKS decide qué entra al armado final. Si no quieres bloques, config.json toma el control vía dashboard_modular.

Cómo construir un nuevo bloque paso a paso

Esta sección explica el proceso completo como si estuviéramos creando un bloque totalmente nuevo desde cero. El objetivo será construir una gráfica distinta: promedio de venta por empresa, ubicada en la fila 3, columna 1.

Paso 1. Define el objetivo del bloque

Primero decide qué quieres visualizar. En este ejemplo, el bloque mostrará el promedio del total agrupado por empresa. Antes de escribir código debes tener claro: qué métrica usarás, qué dimensión la agrupa y en qué celda del layout se colocará.

Paso 2. Elige la celda correcta dentro del layout

El bloque irá en `builder.cell(3, 1)`. Esto significa fila 3, columna 1. Esta celda debe existir en `layout_rows` y no puede estar ocupada por otro widget. Si ya existe otro bloque usando ese espacio, `builder.add(...)` lanzará error.

Paso 3. Crea la función del bloque

Cada bloque nuevo se define como una función con la misma firma estándar: recibe `builder` y `resolver`. `builder` administra el armado del dashboard; `resolver` ayuda a convertir nombres lógicos a columnas reales del CSV.

Paso 4. Resuelve las columnas base

Con `columns = _default_columns(resolver)` obtienes un diccionario con nombres útiles ya mapeados, por ejemplo `columns["empresa"]`, `columns["total"]` o `columns["apertura"]`. Esto evita depender de encabezados exactos del CSV.

Paso 5. Elige la factory adecuada

Como vamos a construir una gráfica de barras, la factory correcta es `bar(...)`, definida en `graficos.py`. Si fuera un KPI usarías `kpi_sum`, `kpi_avg`, etc. Si fuera tabla usarías `tabla(...)`.

Paso 6. Escribe el widget dentro de `builder.add(...)`

Aquí es donde realmente nace el bloque. El widget se crea con su factory y luego se registra con `builder.add(...)`. En este momento también queda reservada la celda para evitar duplicados.

Paso 7. Asigna los valores uno por uno

Ahora llenas cada parámetro del widget: `columna X`, `columna Y`, título, celda, id y configuración extra como `aggregation` o `top_n`. Cada valor tiene un efecto directo en el comportamiento o la vista del bloque.

Paso 8. Registra el bloque en `DEFAULT_BLOCKS`

Después de crear la función, debes agregarla a `DEFAULT_BLOCKS`. Si no haces esto, el dashboard por bloques no la ejecutará aunque la función esté bien escrita.

Paso 9. Prueba el resultado

Genera el dashboard y valida visualmente que el bloque aparezca en la celda correcta, con el título esperado, usando la agregación correcta y mostrando los datos adecuados.

Construcción progresiva del bloque

Vamos armándolo poco a poco para que se vea qué se agrega en cada paso.

1) Estructura mínima de la función

Primero solo definimos la función y los parámetros que siempre debe recibir:

```
def bloque_fila_3_promedio_empresa(builder: DashboardBuilder, resolver: ColumnResolver) -> None:
    pass
```

Aquí todavía no hace nada. Solo declaramos una función válida que luego llenaremos.

2) Resolver columnas base

El siguiente paso es pedir las columnas estándar ya resueltas:

```
def bloque_fila_3_promedio_empresa(builder: DashboardBuilder, resolver: ColumnResolver) -> None:
    columns = _default_columns(resolver)
```

Con esto ya podemos usar valores como `columns["empresa"]` y `columns["total"]` sin depender del nombre exacto del CSV.

3) Crear el widget con la factory bar(...)

Ahora agregamos una gráfica de barras dentro del builder:

```
def bloque_fila_3_promedio_empresa(builder: DashboardBuilder, resolver: ColumnResolver) -> None:
    columns = _default_columns(resolver)
    builder.add(
        bar(
            columns["empresa"],
            columns["total"],
            "Promedio de venta por empresa",
            builder.cell(3, 1),
        )
    )
```

En este punto ya existe el bloque, pero todavía sin id explícito ni ajustes de agregación.

4) Añadir configuración útil del widget

Ahora completamos el bloque con un id estable y parámetros de comportamiento:

```
def bloque_fila_3_promedio_empresa(builder: DashboardBuilder, resolver: ColumnResolver) -> None:
    columns = _default_columns(resolver)
    builder.add(
        bar(
            columns["empresa"],
            columns["total"],
            "Promedio de venta por empresa",
            builder.cell(3, 1),
            id="chart_promedio_empresa",
            aggregation="avg",
            top_n=8,
        )
    )
```

Explicación de cada valor usado:

- `columns["empresa"]`: será el eje X o la dimensión principal.
- `columns["total"]`: será la métrica que se va a agregar.
- `"Promedio de venta por empresa"`: es el título visible del bloque.
- `builder.cell(3, 1)`: indica la ubicación exacta dentro del layout.
- `id="chart_promedio_empresa"`: crea un identificador estable para el widget.
- `aggregation="avg"`: indica que se calculará promedio, no suma.
- `top_n=8`: limita la cantidad de categorías visibles para que la gráfica no se sature.

5) Registrar el bloque para que sí se ejecute

Una vez creada la función, todavía falta conectarla al armado por defecto:

```
DEFAULT_BLOCKS.append(bloque_fila_3_promedio_empresa)
```

Sin esta línea, el bloque existe en el archivo pero no entra al dashboard final.

6) Resultado completo del bloque

Así quedaría el bloque completo y listo para usarse:

```
def bloque_fila_3_promedio_empresa(builder: DashboardBuilder, resolver: ColumnResolver) -> None:
    columns = _default_columns(resolver)
    builder.add(
        bar(
            columns["empresa"],
            columns["total"],
            "Promedio de venta por empresa",
            builder.cell(3, 1),
            id="chart_promedio_empresa",
            aggregation="avg",
            top_n=8,
        )
    )

DEFAULT_BLOCKS.append(bloque_fila_3_promedio_empresa)
```

Resumen mental del proceso

La secuencia correcta siempre es: definir la función -> resolver columnas -> crear el widget con su factory -> ubicarlo en una celda válida -> configurarlo -> registrarlo en DEFAULT_BLOCKS -> generar y validar el dashboard.